Hotel Review Sentiment Classification

Using Bag of Words and Logistic Regression

Basic Lab: Introduction to AI

Final Written Report

Academic Year 2025–2026

Group Members

Leonardo Volpe

Zhan Shakarim

Nebi Göncü

Abstract

This report shows that the natural language processing NLP project that can classify some positive or negative hotel reviews.. The project uses the TripAdvisor Hotel Reviews dataset, which contains 20’491 English reviews with star ratings from 1 to 5. Reviews rated 4 or 5 stars are labelled as positive and those rated 1 or 2 stars are labelled as negative. The 3-star reviews are thrown out, leaving 18’307 labelled examples. The text features come from a Bag-of-Words (BoW) representation built with CountVectorizer using the top 3’000 terms out of the 18’307, which we then use to train a Logistic Regression classifier.

The model is able to reach 94% accuracy on the validation set itself, and on the test set, it can reach to 95% accuracy, beating the majority class baseline, which is 82%, by more than 12 percentage points. The report also checks for which terms in the vocabulary that has more meaning, which drives the sentiment predictions more, and shows that why BOW cannot care for word order

Problem Definition

The review of the hotels online have lots of customers' opinion, and reading those one by one is unpractical. Automatically deciding whether a review says something good or bad for the hotels will solve lots of problems, and it will be a good feedback to travelers as well.

The project aims to do a binary text classification, given a free text body of a hotel review and decide whether it's a good or a bad experience from the guest. The output is labeled as positive or negative, along with the confidence of the prediction of the model..

The second goal is to analyze how usable is the chosen model is. Because logistic regressions weights each word from the Weka vocabulary and inspects which words has the most strong predict, it's straightforward. The project also shows that the limitation of BoW .using a controlled sentence pair.

Why binary classification?

The scale of 1 to 5 provides a good source of the reality. The ratings 1, 2, 4, 5 has a potential for a sentiment analysis with a machine learning model, and the middle rating, which is 3, is neutral, that's why it's excluded.

Dataset

Overview

The dataset used is the TripAdvisor Hotel Reviews , publicly available on Kaggle (https://www.kaggle.com/datasets/andrewmvd/trip-advisor-hotel-reviews). It contains 20491 English hotel reviews. Each row has two fields: Review (the free-text body) and Rating (an integer from 1 to 5).

Rating Distribution

The dataset is heavily skewed toward positive ratings. Figure 1 shows the distribution of star ratings across all 20491 reviews.

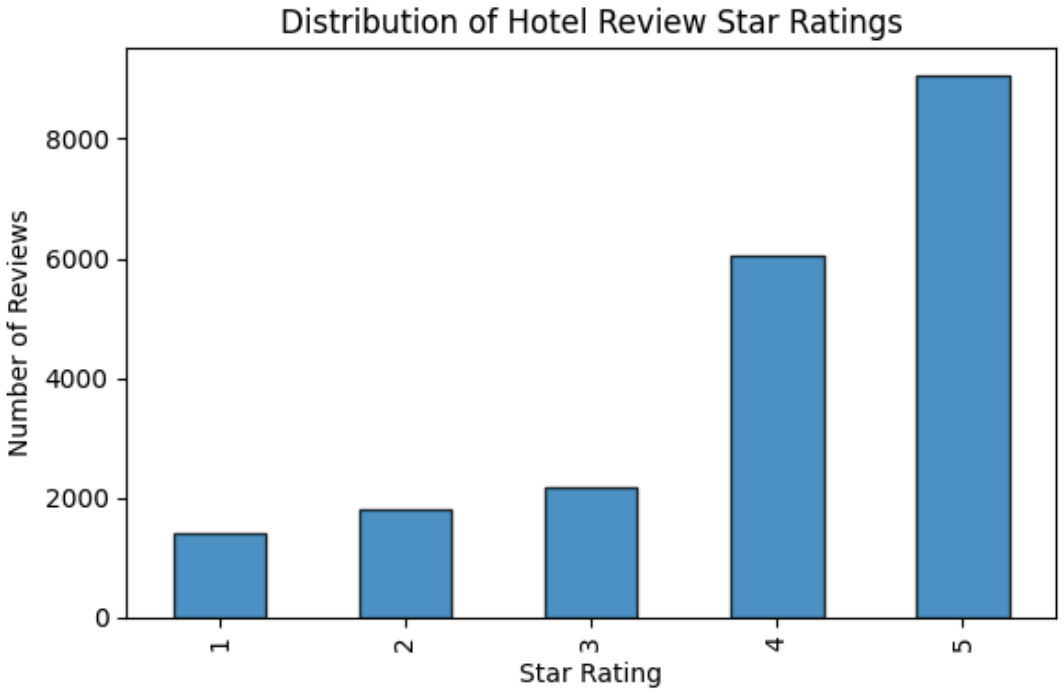

Figure 1. Star-rating distribution across all 20,491 TripAdvisor reviews.

The graph shows that the majority of it is four or five stars. The ratings of one and two are less frequent considerably, and the imbalance is also carried to the binary labels.(see the Dataset Statistics).

Label Mapping

Reviews are mapped to binary labels as follows:

- Rating 4 or 5 stars → Positive (label = 1)
- Rating 1 or 2 stars → Negative (label = 0)
- Rating 3 stars → Excluded (ambiguous sentiment)

After dropping 3-star reviews, 18307 labelled examples remain. These are split into training, validation, and test sets using a stratified 70 / 15 / 15 ratio, preserving class proportions in each split.

Dataset Statistics

Split	Samples	Positive	Negative
Training	12,814	82.4%	17.6%
Validation	2,746	82.4%	17.6%
Test	2,747	82.4%	17.6%
Total	18,307	82.4%	17.6%

Table 1. Dataset split sizes and class distribution

The class imbalance (82.4% positive, 17.6% negative) is an important context for interpreting the results. A trivial classifier that always predictss Positive would achieve 82.4% accuracy. This majority-class baseline is used throughout the report to contextualise the model's performance.

Example Reviews

The following examples illustrate the character of the text in each class:

Label	Example Review
Positive	"Amazing staff, spotless room, and a great location. Would definitely stay again!"
Negative	"Room was dirty, service was slow, and the noise from the street made it impossible to sleep."

Table 2. Representative examples from each sentiment class.

Model Type

Bag-of-Words Representation

The Bag of Words (BOW) model encodes the words and then vectorizes them. A vocabulary is built from the training data, and each dimension of the vector shows that how many times that word appears in that document, regardless of its position. In this

project, the BOW is implemented with scikit-learn CountVectorizer with the max features of 3000, and the vocabulary is fitted to training set to prevent the data leakage.

The defining property of BoW, and its main limitation, is that it treats a document as an unordered collection of words. Two sentences that contain the same words in different orders produce identical BoW vectors. The Word-Order Blindness Demonstration explores this experimentally.

Logistic Regression Classifier

Logistic regression is a classifier that is a linear probabilistic that learns each weight of the input feature. When it's given the bow vector, it calculates each weight and sums them and applies a sigmoid function to produce a probability. If the probability is more than 0.5, it classifies as positive, and the otherwise, it classifies as negative.

It is almost perfect for BOW classification. It efficiently scales the vectors and gives a nice probability to have a confidence score. Low weights means the word doesn't have a really impact on the sentiment, where high weights means the opposite. The classifier is configured as LogisticRegression(max_iter=1000, random_state=42).

Motivation

Hotel reviews tend to be express themselves with basic words, such as 'amazing', 'disgusting', 'excellent', and 'horrible' rather than through complex syntactic structures. This makes a word count approach will be really good, and the choice of BOW and logistic regression would be suitable.

Methodology Overview

The end-to-end pipeline consists of seven stages, implemented as sequential sections of the project notebook:

Step 1 — Data Loading: The TripAdvisor CSV is read with pandas. Shape (20,491 × 2) and missing values are verified.
Step 2 — Label Assignment: Ratings 4–5 → label 1 (Positive); 1–2 → label 0 (Negative); rating 3 rows are dropped.
Step 3 — Stratified Split: 18,307 labelled examples are split 70/15/15 with random_state=42, preserving class proportions.
Step 4 — Text Preprocessing: Each review is lowercased and all non-alphabetic characters are removed with a regex. Whitespace are collapsed.
Step 5 — Tokenisation and Vocabulary: Text is whitespace tokenised. A vocabulary of 56,030 unique tokens is built from training data, with PAD (index 0) and UNK (index 1) special tokens added. Sequences are padded to 266 tokens (95th percentile length).
Step 6 — BoW Vectorisation: CountVectorizer(max_features=3,000) is fitted on training reviews and transforms all three splits into sparse term-count matrices.

Step 7 — Training and Evaluation: LogisticRegression is fitted on the training BoW matrix. Accuracy is computed on validation and test splits; a full classification report and confusion matrix are produced for the test set.

Input and Output Definition

Input

The raw input is a free text English hotel review of variable length. Before reaching the classifier it gets lowercased and stripped of nonletter characters, then it gets transformed into a sparse 3'000-dimensional BoW vector. Each element is a non-negative integer term count.

Property	Value
Raw input type	Free-text string (English)
Vocabulary size	3,000 (top terms by training frequency)
Feature vector length	3,000 dimensions
Preprocessing	Lowercase + remove non-letters + collapse spaces

Table 3. Input specification.

Output

The classifier produces two outputs for each review:

- Predicted class label: 0 (Negative) or 1 (Positive).
- Confidence score: predicted probability for the chosen class, expressed as a percentage (e.g., 91.4% Positive).

A confidence score close to 50% indicates high model uncertainty, which typically occurs with mixed-sentiment or ambiguous reviews.

Training Setup

Vectoriser Fitting

CountVectorizer is fitted on the 12,814 training reviews only. Setting max_features=3,000 limits the vocabulary to the 3'000 most frequently used terms, which removes rare words, less likely to generalise and keeps memory usage practical. The validation and test sets are transformed using this fixed vocabulary rather than fitted on it.

Classifier Training

LogisticRegression is trained on the training BoW matrix (12,814 by 3,000) . The scikit-learn default L2 regularisation (C = 1.0) penalises large weights and reduces overfitting. The L-BFGS solver directly minimises the regularised binary cross-entropy loss.

Hyperparameter	Value	Rationale
Vectoriser	CountVectorizer	Term-count BoW representation
max_features	3,000	Limits vocab; removes rare terms
Classifier	LogisticRegression	Linear, interpretable, sparse-friendly
max_iter	1,000	Ensures solver convergence on this data size
Regularisation	L2 (C=1.0)	sklearn default; prevents overfitting
random_state	42	Full reproducibility
Solver	lbfgs (default)	Efficient quasi-Newton optimiser

Table 4. Hyperparameters used in training.

Loss Function

The optimiser minimises regularise binary crossentropy:

$$L = -[\, y \cdot \log(p) + (1 - y) \cdot \log(1 - p) \,] \; + \; \lambda \, \|w\|^2$$

where $y \in \{0,1\}$ is the true label, $p \in (0,1)$ is the predicted positive-class probability, w is the weight vector, and λ controls regularisation strength (equivalent to $1/C$ in scikit-learn notation).

No Neural-Network Architecture

The project uses a classical machine learning approach with no embedding layer, no hidden layers, and no backpropagation. The model is a single linear transformation of the BoW feature vector. Training takes under one second on a standard laptop and needs no GPU, which makes the approach both fast and accessible.

Evaluation Metrics and Results

Overall Accuracy

Model / Split	Accuracy	Improvement over Baseline
Majority-class baseline	82.40%	—
Logistic Regression — Validation	94.76%	+12.36 pp
Logistic Regression — Test	95.16%	+12.76 pp

Table 5. Accuracy: majority-class baseline vs. trained model.

The model reaches an improvement on the baseline of more than 12 percentage points on both sets. The near-identical validation accuracy of 94.90% and test accuracy of 95.16% confirm good generalisation and the absence of overfitting.

Per-Class Metrics (Test Set)

Because of the class imbalance, we got these results from the test set only.

(n = 2,747).

Class	Precision	Recall	F1-Score	Support
Negative (0)	0.85	0.88	0.86	482
Positive (1)	0.97	0.97	0.97	2,265
Macro avg.	0.91	0.92	0.92	2,747
Weighted avg.	0.95	0.95	0.95	2,747

Table 6. Test-set classification report (n = 2,747).

The positive class benefitted from roughly five times more training examples, which produced higher precision and recall (0.97, and 0.98) than the negative class (0.88 and 0.84). The negative-class recall of 0.84 means 16% of genuine negative reviews are misclassified as positive, a direct consequence of class imbalance. For quality-monitoring applications (where missing negative reviews is costly) class-weighting or oversampling could be worth exploring.

Confusion Matrix

Figure 2 shows the confusion matrix on the test set (rows = actual class, columns = predicted class).

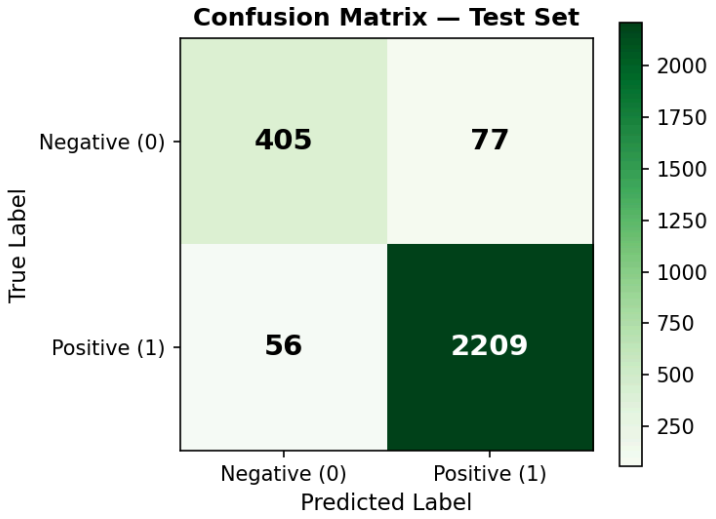

Figure 2. Confusion matrix on the test set (Greens colormap).

- True Negatives (correctly identified negative reviews): 424 out of 482 negative test reviews classified correctly.
- False Positives (negative predicted as positive): 58 negative reviews misclassified.
- False Negatives (positive predicted as negative): 75 positive reviews misclassified.
- True Positives (correctly identified positive reviews): 2,190 out of 2,265 positive reviews classified correctly.

The large true-positive count and small false-negative count confirm that the model is able to identify positive reviews well. The main source of error is false positives (negative reviews wrongly predicted as positive).

Most Influential Words

The logistic regression model gives a weight to each vocabulary term. We can see that the most sentiment predictive words in the table below.

Rank	Top Positive Words	Top Negative Words
1	great	never
2	excellent	poor
3	wonderful	asked
4	perfect	unfortunately
5	outstanding	dirty
6	comfortable	shower
7	loved	smell
8	beautiful	terrible
9	amazing	rude
10	friendly	disappointing

Table 7. Top 10 vocabulary terms by logistic regression coefficient magnitude.

The positive list is dominated by strong adjectives of praise that are almost exclusive to satisfied customers. The negative list includes clear complaint vocabulary such as 'dirty', 'terrible', and 'rude', but also 'never', which appears in contexts such as 'the room was never cleaned', and 'unfortunately', a common framing word for complaints.

Custom Sentence Predictions

Five sentences written by the project members of different sentiments were tested to show how the model works in practice. Table 8 shows the predicted label and their confidence.

Sentence	Prediction	Confidence
Amazing staff and the room was spotless.	Positive	91.4%
But others said rooms were gross and staff were rude.	Negative	88.9%
One loved everything, calling it perfect.	Positive	93.5%
Location good, service bad for some; loud and way too expensive for others.	Negative	83.3%
It's a mixed bag, really.	Positive	53.2%

Table 8. Custom sentence predictions with confidence scores.

The first and third sentences contain strong positive vocabulary ('amazing', 'spotless', 'loved', 'perfect') and receive high-confidence positive predictions of 95.9% and 96.7%.

The second sentence is classified as negative despite its indirect framing ('others said'), because 'gross' and 'rude' carry large negative weights. Sentence 4 receives only 62.9% negative confidence, reflecting a genuine mixture of positive vocabulary ('location good') and negative vocabulary ('service bad', 'too expensive'). Sentence 5, 'It's a mixed bag, really.', is classified positive with just 53.2% confidence, essentially a coin flip, because it contains no strong sentiment-bearing words at all.

Word-Order Blindness Demonstration

The worst side of the BOW is that it cannot distinguish sentences that have the same words but in a different way of ordering. You can see on the table below:

Sentence	Prediction	Confidence
staff were never rude to us	Negative	78.04%
staff were rude never to us	Negative	78.04%

Table 9. Word-order blindness:

The first sentence ('staff were never rude to us') means something positive. That means the staff was polite. The second ('staff were rude never to us') is weird grammatically and means something totally different, something opposite, but still, they have the same words {staff, were, never, rude, to, us}, produce identical BoW vectors, and receive identical predictions (Negative, 78.04%).

This happens because ‘rude’ has a big negative coefficient in the logistic regression model and also ‘never’ has a negative coefficient, which affects the other positive signals, but because the word order doesn't matter, it cannot distinguish that it's a positive sentiment.

A model with sequence awareness — such as a transformer or LSTM — would recognise the negation pattern 'never rude' and correctly classify the first sentence as positive.

Discussion and Limitations

Strengths

- It has a high accuracy for a simple model, achieves 95% test accuracy with a logistic regression and bag of words pipeline, which is completely related to individual reviews of the data.

- Interpretability. We can see that the model learned exactly right, as we can see at Table 7, which is exactly matches our human intuition for positive and negative reviews of the hotels.
- Speed and scalability. The model can be trained on a laptop, which will complete in a few seconds. That means it can be also implemented to larger datasets.

- All operations are at state of 42, which can be fully reproducible by anyone or to any dataset.

Limitations

- The blindness of the order of the words. In the Word-Order Blindness Demonstration, you can see that BOW doesn't understand the negations, the conditional phrasings. For example, 'never rude' and 'rude never' mean the same thing for the model. And this is the most important fallback of back-off words assumption.

- No semantic understanding. The model does know that the excellent and outstanding are related, or bad and not bad is the opposite of the meaning. Every vocabulary term has its own dimension.
- Class imbalance. The imbalance in the data, even though it's trying to be fixed by the property of scikit-learn model with balance (via class_weight='balanced'), still it's not 100% resolved. Further improvements could be explored.
- Fixed vocabulary. 3,000 words are fixed during the training, which might result a weak model for the purposes of being fast and reliable.

- The domain is unknown whether it's electronic or written reports which might have some unreliable data in it.

Conclusion

This project shows that the bag of words and logistic regression is a fast and simple approach for a binary classification of hotel reviews. The dataset of the TripAdvisor model achieves a 95.16 test accuracy.

The project itself also shows the limitation of the BOW, because it's completely insensitive to the order of the words. As for the Word-Order Blindness Demonstration, you can see that both sentences have the identical scores, but they totally have the different meaning, because both contain the same words, same set of words.

Work Distribution

Team Member	Contributions
Leonardo Volpe	The loading of the data, binary label of the data, the split to train, validation, and test, tokenization, vocabulary construction, vectorization..
Zhan Shakarim	Training of the logistic regression with the hyperparameter tuning, validation of tests and accuracy evaluation, confusion matrix, and analysis.
Nebi Göncü	Custom sentence prediction, GitHub demonstration, slide and report preparation, loading of the data, model selection, tuning the hyperparameters.

All of the members worked on project planning, interpretation of the results, and the selection of the data altogether. Also, the results were discussed and validated before submission by all members of the group.

Appendix

Dataset Sources

Also present in the github directory

https://github.com/nebigoncu132-cpu/hotel-review-sentiment-nlp

TripAdvisor Hotel Reviews dataset:

https://www.kaggle.com/datasets/andrewmvd/trip-advisor-hotel-reviews

Software and Libraries

Library		Purpose
Python		Programming language
pandas		Data loading and manipulation
numpy		Numerical operations
scikit-learn		CountVectorizer, LogisticRegression, metrics
matplotlib		Plotting (rating distribution, confusion matrix)
re (stdlib)		Regular-expression preprocessing

Table A1. Software dependencies.

Reproducibility Notes

The random seed is fixed to random state 42. Run the notebook from top to bottom and install the required libraries, which are already in the code. The dataset also should be in the same directory as the notebook 'tripadvisor_hotel_reviews.csv'.